Enterprise Employee Analytics Data Warehouse System

Project Technical and User Documentation

This documentation explains the application, its operational and analytical data models, the synthetic data pipeline, user workflows, and the steps required to run the system locally.

Document detail	Value
Document version	1.0
Issue date	31 October 2026
Document type	Formal project documentation
Intended readers	HR users, analysts, database administrators, and project reviewers
Project scope	Local Streamlit application, MySQL schemas, synthetic data generation, and analytics

The implementation combines a Streamlit user interface with Python services and three MySQL schemas. Its operational database manages employees, projects, assignments, and reviews; its warehouse keeps employee history and supports review and workforce analysis.

How to Use This Documentation

Use the architecture and data-model sections to understand how records move from the source dataset through staging and operational storage into the analytics warehouse. Use the application guide for day-to-day HR tasks, and use the setup and operations sections when installing, loading, or diagnosing a local instance.

Reading Map

Section	What it covers
System purpose and scope	Users, capabilities, and project boundaries.
System architecture	Application layers and the path from source data to analytics.
Operational data model	OLTP entities, keys, relationships, and constraints.
Staging data model	Batch tracking and landing tables used by the loader.
Analytics data model	Star-schema dimensions, review fact grain, and employee history.
Data pipeline	Synthetic generation, SQL loading, ETL order, and SCD Type 2 handling.
Application guide	Employee, project, review, home, and dashboard workflows.
Analytics and controls	Metric definitions, validations, privacy, and known limitations.
Installation and operations	Prerequisites, local setup, recurring procedures, and troubleshooting.

Audience and Responsibilities

Reader	Primary use
HR user	Maintain employee records, assign project work, and record performance reviews.
Analyst	Interpret the descriptive dashboard metrics and review trends.
Database administrator	Create schemas, configure credentials, run load procedures, and review batch status.
Project reviewer	Understand the system architecture, entity relationships, and warehouse design.

Data notice The loader expands the IBM HR attrition source into synthetic employee, project, assignment, review, and history records. Generated trends are demonstration data and must not be treated as actual workforce outcomes.

1 System Purpose and Scope

The Enterprise Employee Analytics Data Warehouse System is a local employee-record and workforce-analytics application. HR users can onboard and update employees, preserve selected employment changes as history, create projects, assign employees, and record performance reviews. Analysts can compare review scores, employee attrition labels, project delivery dates, and assigned capacity.

The application is implemented in Python with Streamlit. MySQL stores operational records and the analytics warehouse. Python managers coordinate database operations; SQL procedures load staging data into the normalized operational schema and then build the warehouse.

Functional Scope

Capability	System behavior
Employee records	Onboard, search, edit selected fields, and display employment history.
Projects	Create projects with planned or active status, view project status, and assign employees.
Assignments	Record dates, billable status, and allocation percentage; warn when combined allocation exceeds 100%.
Performance reviews	Record a review date, 1-5 rating, 0-100 score, status, optional project, and comments.
Analytics	Show current headcount, average review score, attrition status rate, review volume, rankings, and bottlenecks.
Data warehouse	Maintain date, department, project, and employee dimensions and a review fact table.

System Boundaries

The repository defines a local application and documentation invoked data-load process. It does not define an automated job scheduler, a production identity provider, or a role-based authorization workflow. The supplied analytics are descriptive; the code does not train or serve an employee attrition prediction model.

The HR source data does not contain real performance-review events or a complete sequence of job changes. Review events and selected employment history are generated for the demonstration pipeline.

2 System Architecture

The system separates user interaction, domain operations, database connectivity, data ingestion, and warehouse processing. This separation lets Streamlit pages call Python managers rather than embedding database statements in every form.

Layer	Project components	Responsibility
User interface	app/streamlit_app.py and app/pages/	Home analytics, employee records, projects, reviews, and the analytics dashboard.
Shared helpers	app/helpers.py and app/styles.py	Employee search widgets, display labels, page headers, and chart styling.
Domain services	src/managers.py and src/entities.py	Employee, project, review, and analytics operations; entity validation.
Database access	src/db_manager.py and src/config.py	MySQL connection pool, SQL execution, environment configuration, and database errors.
Data ingestion	run_staging_load.py and src/etl/	Synthetic data generation and bulk loading into staging tables.
Persistence and warehouse	sql/00 through sql/07	Schema creation, operational loads, dimensions, review fact, and the full ETL procedure.

Request and Data Flow

1. An HR user opens a Streamlit page and submits a form or analytics filter.
2. The page validates inputs through the entity or service layer and calls a manager.
3. The database manager obtains a connection from the shared MySQL pool, executes SQL, commits successful work, and returns rows or a controlled database error.
4. Separately, the staging runner uses Pandas, NumPy, and Faker to produce synthetic datasets and writes them to hr_staging in chunks.
5. SQL load procedures move employee and activity records into hr_oltp. The warehouse ETL populates hr_dw dimensions and review facts.
6. Analytics managers query hr_dw for workforce and review measures; capacity and overdue-project views also query operational data.

Architecture Diagram

The supplied architecture diagram shows the complete path: HR pages call shared helpers and domain managers; managers use validated entities and database access; the staging runner writes generated datasets to the staging schema; SQL loaders populate the operational schema; and ETL procedures build the analytics warehouse.

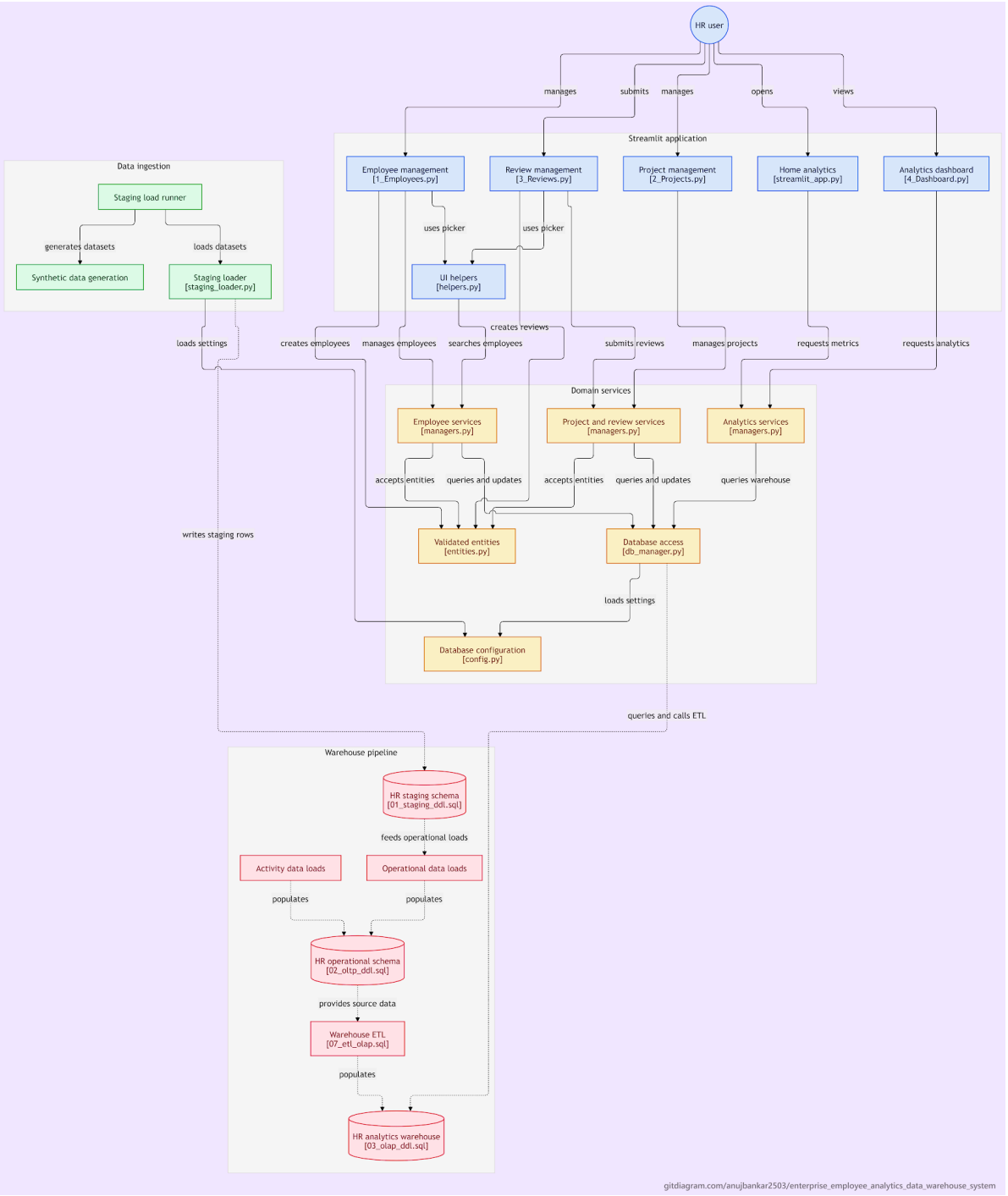


Figure 1. Application, ingestion, operational database, and warehouse flow.

3 Operational Data Model

The hr_oltp schema is the normalized operational store. Departments and projects are maintained as their own entities. Employees reference departments. Assignments resolve the many-to-many relationship between employees and projects. Reviews belong to an employee and may optionally reference a project and a reviewer.

Entity	Primary information and relationships
departments	Department ID, unique name and code, active flag, and creation timestamp. One department can contain many employees.
employees	Employee ID and unique employee number; name, hire date, department, role, level, location, income, demographic and work attributes, and attrition label.
projects	Project ID and unique code; name, lifecycle status, start, planned end, and actual end dates.
assignments	Assignment ID; employee and project foreign keys; start/end dates, billable flag, and allocation percentage.
reviews	Review ID; reviewed employee, optional reviewer and project, date, 1-5 rating, 0-100 score, status, and optional comments.

Relationships

- A department has many employees; each employee references one department.
- An employee can have many assignment records, and a project can have many assignments. The assignments table implements the employee-to-project many-to-many relationship.
- An employee can receive many reviews. A reviewer is an employee foreign key and may be null. A review may be associated with a project or left unassigned.

Primary keys are generated numeric identifiers. Employee numbers, department names and codes, and project codes provide unique business identifiers. Foreign keys protect employee, project, reviewer, and department references. SQL checks constrain income, age, education, working years, company count, allocation, rating, and score.

Operational ER Diagram

The original ER diagram names each operational table and marks its primary and foreign keys. Its relationship lines show one-to-many department-to-employee and employee/project-to-assignment relationships, plus the employee-to-review relationships.

OLTP ER Diagram (schema: hr_oltp)

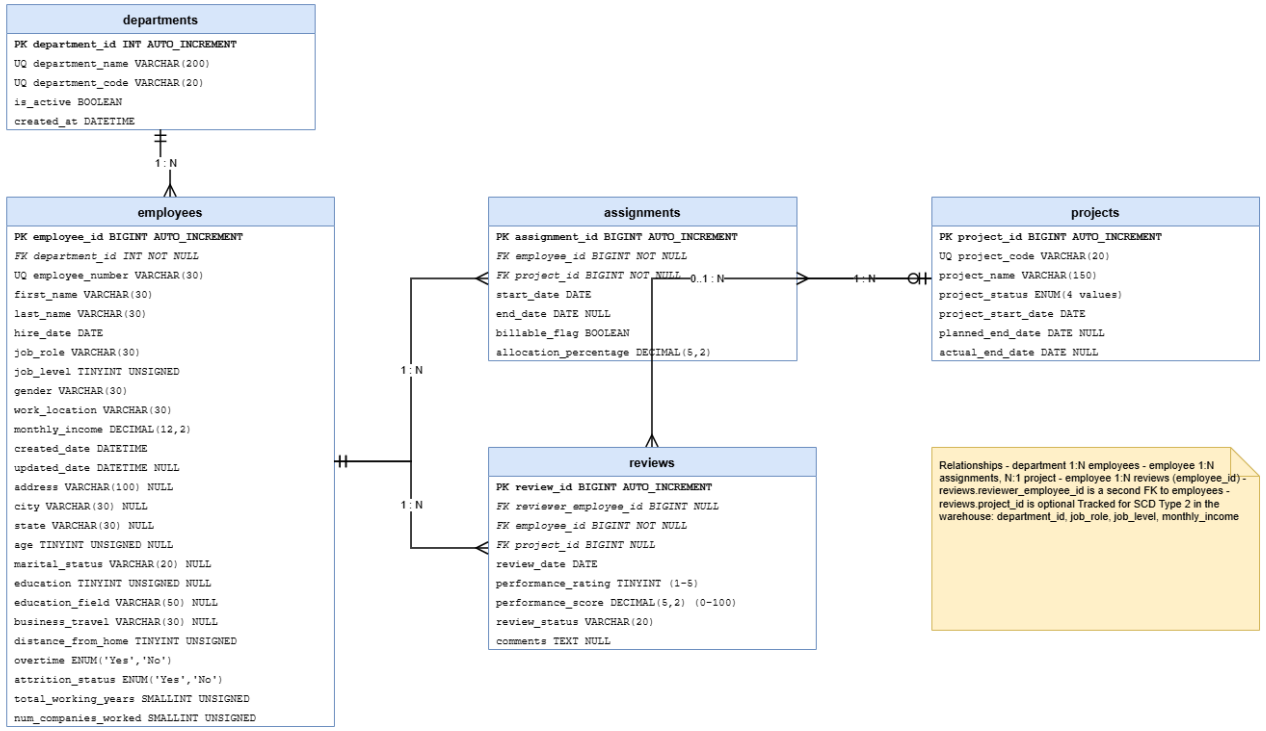


Figure 2. Operational entity relationships in hr_oltp.

4 Staging Data Model

The hr_staging schema is the landing area for each generated load. Every staging row carries a batch ID, and the ETL batch log records the source name, start and completion times, status, rows read, rows loaded, rejected rows, and any error text.

Staging table	Purpose and key contents
etl_batch_log	One row per load run; operational status and row-count audit fields.
stg_hr_employee	Source-derived employee attributes, generated employee number and names, hire date, department, job attributes, pay, and other HR fields.
stg_employee_history	Effective-dated department, role, level, and income versions with current-row flag and change reason.
stg_project	Generated project code, name, status, start, planned end, and actual end dates.
stg_assignment	Generated employee/project link, assignment dates, billable flag, and allocation percentage.
stg_review	Generated employee/project review event, date, rating, score, status, and optional comments.

The staging diagram documents a 100,000-row employee generation target. The source dataset is identified as the IBM HR attrition CSV, with approximately 1,470 source employees; the loader samples with replacement and adds generated identifiers and names before inserting the enlarged dataset.

Staging Schema Diagram

The supplied staging diagram shows the batch log as the parent of the employee, history, project, assignment, and review landing tables. It also calls out the synthetic-data source and 100,000-row employee target.

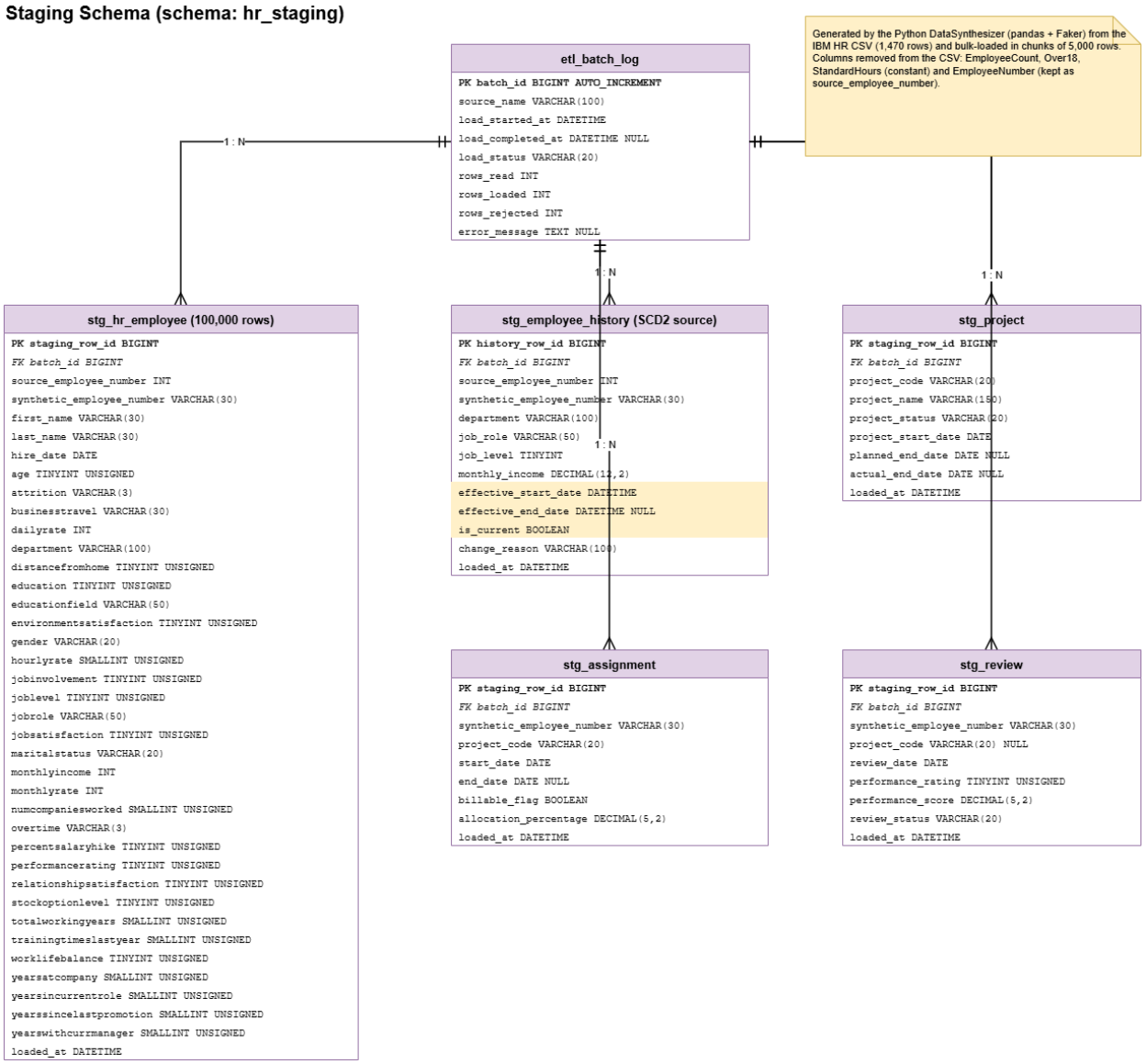


Figure 3. Batch-tracked landing tables in hr_staging.

5 Analytics Data Model

The hr_dw schema uses a star design for review analysis. The fact table has one row per review record. Each fact links to the employee version valid on the review date, the review date, the employee's department version, and either a project or the unknown-project member.

Warehouse table	Role in analysis
dim_employee	Employee profile with a surrogate employee key, business employee ID, and effective start/end dates. SCD Type 2 keeps historical versions.
dim_department	Department lookup with a surrogate key and operational department ID. Loaded as Type 1.
dim_project	Project lookup with a surrogate key and operational project ID. Loaded as Type 1; key -1 represents an unknown or unassigned project.
dim_date	Calendar lookup keyed as YYYYMMDD and populated for dates from 2020 through 2030.
fact_performance_reviews	One row per review ID; stores dimension keys, rating, score, and review count of one.

Fact Table Grain and Measures

The fact grain is a single operational review. The review ID is retained as a unique degenerate key. Rating and score are the principal measures; review_count is set to one so events can be counted by dimension. The fact loader inserts only review IDs not already present.

Analytics Star Schema Diagram

The original star-schema diagram places fact_performance_reviews at the center and connects it to employee, project, date, and department dimensions. It highlights the Type 2 validity columns in dim_employee and notes that review dates select the employee version valid at that time.

OLAP Star Schema (schema: hr_dw)

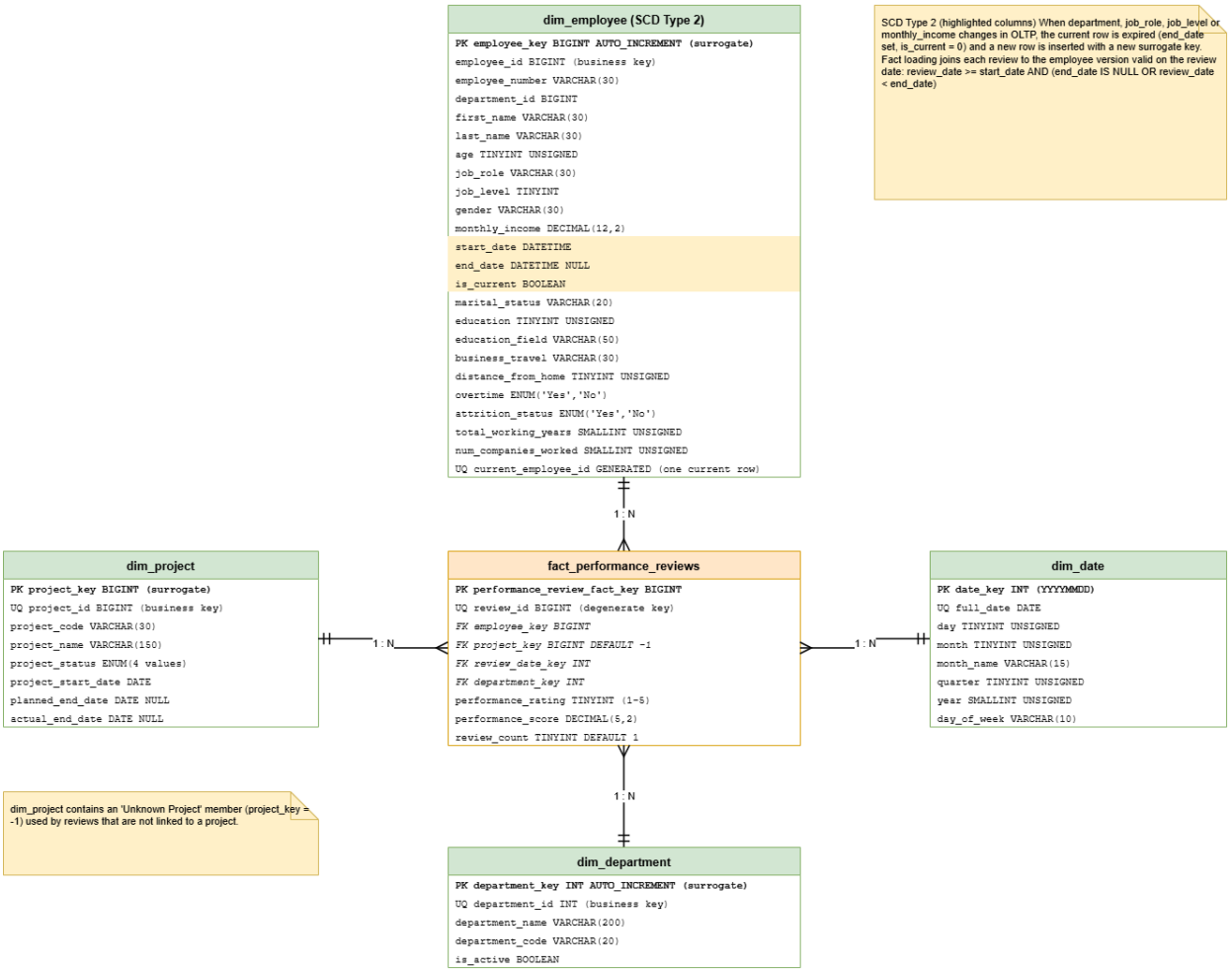


Figure 4. Review-event star schema in hr_dw.

6 Data Generation and Warehouse Load

The repository's run_staging_load.py creates a reproducible demonstration dataset from the IBM HR source CSV. DataSynthesizer uses a fixed random seed, Pandas, NumPy, and Faker. The default employee scale is 100,000.

Generated Datasets

- Employees are sampled with replacement from the source. The pipeline adds unique EMP-number identifiers, Faker names, and bounded variation in age, income, and experience.
- Hire dates are derived from company tenure. For eligible employees, the history generator creates effective-dated promotion, salary-change, or department-transfer versions.
- The generator creates 300 projects with Planned, Active, Completed, or Cancelled status.
- Employees receive one to three project assignments where generated dates are valid. Assignments include billable status and a percentage allocation.
- One to three review events are generated per eligible assignment. Review ratings and scores are generated for demonstration; some reviews are not linked to a project.

Load Sequence

7. The staging runner reads the source CSV, removes constant source columns, generates employee and activity datasets, and connects to hr_staging.
8. The staging loader truncates existing staging tables, creates an ETL batch log row, and loads each dataset in chunks of 5,000 rows.
9. sql/05_load_oltp_employees.sql inserts departments and one employee row per synthetic employee number. A window function selects the latest duplicate staging row.
10. sql/06_load_oltp_activity.sql loads projects and, when their target tables are empty, assignments and reviews. Project references on reviews are optional.
11. sql/07_etl_olap.sql creates the dimension and fact procedures. CALL hr_dw.sp_run_full_etl() loads the calendar, department, project, employee, and review fact tables in sequence.

Rerun behavior run_staging_load.py truncates staging tables before loading a new batch. Preserve or back up staging data before rerunning if the current landing rows are required for audit or recovery.

Application Changes

Employee creation, tracked employee updates, project creation, and review submission call the warehouse synchronization path from the Python managers. Assignments remain operational records; the dashboard's capacity query reads assignments from hr_oltp.

7 Employee History with SCD Type 2

Slowly Changing Dimension Type 2 preserves prior values instead of overwriting a tracked employee profile. The warehouse assigns a new surrogate employee_key to each version while keeping employee_id as the stable business key.

Tracked Attributes

Tracked attribute	Versioning behavior
Department	A changed department closes the current warehouse row and inserts a new version.
Job role	A changed role creates a new employee dimension version.
Job level	A changed level creates a new version; the procedure uses null-safe comparison.
Monthly income	A changed income creates a new version.

When one of these attributes changes in hr_oltp, the current dimension row receives an end timestamp and is_current becomes false. A new current version is inserted. A generated current_employee_id column is unique for current rows, enforcing at most one current version per employee.

The fact load joins each review to the employee version whose start and end dates contain the review date. This allows historical review analysis to use the profile attributes valid when the review occurred rather than the employee's latest profile.

Source History and Application History

Initial synthetic history is staged for a subset of employees and loaded when their dimension records are first created. Later changes made through employee editing are handled by the warehouse synchronization procedure. The change history covers the attributes tracked by the SCD process; it is not a complete audit trail of every employee field or user action.

8 Application User Guide

The Streamlit application is organized into a home page and four sidebar pages. Forms are submitted explicitly; successful employee and project operations generate identifiers or confirmation messages.

Page	User workflow
Home analytics	Review current headcount, mean review score, attrition status rate, review count, yearly score trend, and department averages.
Employees	Onboard a person, search by employee number or name, edit supported fields, and inspect employment history.
Projects and assignments	Create a project, select an employee, assign the employee to an open project, and inspect project status.
Performance reviews	Select an employee, optionally associate a project, enter date, status, rating, score, and comments, then review recent entries.
Analytics dashboard	Filter review analysis by year and department; inspect performance, top performers, attrition patterns, and project bottlenecks.

Employee Records

Employee onboarding requires first and last name, active department, job role, job level, income, hire date, age, gender, work location, marital status, education, education field, business travel, distance from home, overtime, total working years, and number of prior companies. Employee numbers are generated after submission.

The edit workflow supports department, job role, job level, and monthly income as tracked fields. The employee search helper accepts an employee number or first or last name. The history view displays effective dates, department, role, level, and income.

Projects and Assignments

Project creation collects a unique project code, name, start date, planned end date, and an initial Planned or Active status. Assignment requires an employee, a Planned or Active project, start date, optional end date, billable selection, and allocation percentage. The service rejects allocation outside 0-100% or an end date earlier than the start date. It warns when combined not-ended assignments exceed 100%.

Performance Reviews

A review must reference an employee. A project is optional. Users enter a review date, Completed or Pending status, a 1-5 rating, a 0-100 score, and optional comments. Recent review records for the selected employee appear beneath the form.

9 Analytics and Interpretation

The dashboard supports descriptive analysis of the loaded workforce snapshot and review events. The measures below describe how the SQL managers calculate each result; they do not imply causation or individual employee risk.

Measure or view	Definition in this implementation
Current headcount	Count of dim_employee rows where is_current = 1.
Average review score	Average performance_score in fact_performance_reviews, on a 0-100 scale.
Attrition rate	100 times the average of the current employee attrition_status indicator. This is a snapshot label rate, not time-based turnover.
Recorded reviews	Count of rows in fact_performance_reviews.
Year-over-year score change	Annual average score minus the previous available year's average, calculated with SQL LAG.
Top performers	Average review score ranked within department, subject to a minimum review-count filter; equal scores share a dense rank.
Attrition patterns	Current employee attrition labels grouped by overtime, job level, travel, marital status, job role, or experience band.
Delivery bottlenecks	Active projects past planned end date and employees whose not-ended assignment allocations total more than 100%.

Interpretation Limits

- The dataset is generated from a source snapshot. Review dates, scores, projects, assignments, and selected employee history are synthetic.
- The attrition metric is not calibrated as a predictive probability and does not measure departures over a defined reporting period.
- The score trend is a software demonstration, not evidence of actual employee performance.
- The warehouse fact table is review-centered. Project capacity is calculated from operational assignments rather than a separate assignment fact table.
- The database stores income as a decimal but does not define a currency code. Confirm the intended currency before using amounts for business decisions.

10 Validation and Data Controls

Validation is applied in Python entities and MySQL constraints. Manager methods also check selected business rules before writing records.

Area	Rule
Employee text	First and last names are required and limited to 30 characters; job role and work location are also limited to 30 characters.
Employee numeric fields	Income must be nonnegative and no greater than 10,000,000 in the entity; job level is 1-5; age is 18-100 when supplied; education is 1-5 when supplied.
Project	Project code is unique and limited to 20 characters; name is limited to 150; status is Planned, Active, Completed, or Cancelled; planned end cannot precede start.
Assignment	Allocation is constrained to 0-100. The manager rejects an end date earlier than the assignment start date.
Review	Review date is from 2020-01-01 through 2030-12-31; rating is 1-5; score is 0-100; status is Pending or Completed; self-review is rejected when a reviewer is provided.
Database references	Foreign keys require valid department, employee, project, reviewer, and warehouse dimension keys.
ETL batch	Batch status and row counters are stored in etl_batch_log; load errors can be recorded in error_message.

The operational schema includes database CHECK constraints for numeric ranges and unique constraints for business identifiers. The manager and entity layers should remain enabled so users receive readable validation messages before the database rejects invalid values.

11 Local Installation and Database Setup

The project requires Python 3.10 or later, MySQL 8 or later, the project dependencies, and the IBM HR source CSV at data/WA_Fn-UseC_-HR-Employee-Attrition.csv. The source CSV is not tracked in the repository and must be placed in that location before running the staging loader.

Install Python Dependencies

Run the following command from the project root:

```
py -3 -m pip install -r requirements.txt
```

Configure the Local Database Account

Create app/.env and set the connection values. Use a dedicated MySQL account with access to hr_staging, hr_oltp, and hr_dw. Do not place real credentials in screenshots, source control, or in this documentation.

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=<local_mysql_user>
DB_PASSWORD=<local_mysql_password>
```

Create Schemas and Load Data

Order	Action
1	Run sql/00_create_schemas.sql with a MySQL administrator to create hr_staging, hr_oltp, and hr_dw.
2	Run sql/01_staging_ddl.sql, then sql/04_staging_extra_ddl.sql.
3	Run sql/02_oltp_ddl.sql and sql/03_olap_ddl.sql.
4	Run sql/07_etl_olap.sql to create the warehouse load procedures.
5	Run py -3 run_staging_load.py from the project root. It generates 100,000 employees and related activity, then loads staging.
6	Run sql/05_load_oltp_employees.sql, then sql/06_load_oltp_activity.sql.
7	Call hr_dw.sp_run_full_etl() to populate or refresh the warehouse.

```
mysql -u <DB_USER> -p hr_dw -e "CALL sp_run_full_etl();"
```

Start the Application

```
py -3 -m streamlit run app/streamlit_app.py
```

Run Streamlit from the project root so imports resolve correctly. Open the local address printed by Streamlit in the browser. The application reads database settings from app/.env through src/config.py.

12 Routine Operations

Review a Load

After a staging run, review the corresponding etl_batch_log row. Confirm that load_status is SUCCESS, compare rows_read and rows_loaded, and check error_message if the status is FAILED. The loader commits chunks of 5,000 rows; a failed run can therefore have rows loaded before failure, even though the batch is marked failed.

Refresh Analytics

The dashboard's cached analytics functions use a five-minute cache. Successful create and update actions clear Streamlit's cached data. When data is loaded directly through SQL, rerun the page or wait for the cache window before interpreting the displayed metrics.

Back Up and Rerun Safely

- Back up the MySQL schemas before major reloads or SQL procedure changes.
- The staging runner truncates its landing tables before loading. Preserve a copy of the current batch if it is needed for audit or recovery.
- The employee load avoids duplicate synthetic employee numbers. The activity loader inserts new projects and loads assignments/reviews only when those target tables are empty.
- The warehouse fact loader uses review_id uniqueness to avoid inserting the same review twice.
- Check target-table state and staging truncation behavior before repeating a full load.

13 Troubleshooting

Symptom	Checks and corrective action
Missing database configuration	Confirm app/.env exists and includes DB_HOST, DB_PORT, DB_USER, and DB_PASSWORD. Check spelling and accidental spaces.
MySQL access denied	Confirm the MySQL service is running, credentials are correct, and the account can access all three schemas. Test the account in a local MySQL client.
MySQL connector import error	Install dependencies from the project root with py -3 -m pip install -r requirements.txt using the same Python environment that runs Streamlit.
Source CSV not found	Place WA_Fn-UseC_-HR-Employee-Attrition.csv under data/ or update the path in run_staging_load.py.
No ETL procedure installed	Run sql/07_etl_olap.sql in hr_dw, then call sp_run_full_etl(). Confirm the MySQL account can create and execute stored procedures.
Foreign-key or duplicate-key failure	Check load order, confirm employee and project business keys match staging references, and inspect whether target activity tables already contain rows.
No review trend appears	Confirm reviews were loaded, the warehouse fact table contains rows, and the selected year or department has scored review events.
Unexpected stale metrics	Rerun Streamlit after direct SQL loads; the dashboard caches analytical queries for up to five minutes.
Invalid form submission	Check the field ranges in Section 10. Review dates must fall within 2020-2030; ratings and scores must use their defined scales.

14 Security, Privacy, and Limitations

Credential and Data Handling

The application reads local database credentials from app/.env. Keep that file outside version control and restrict it to the local user account. Use a dedicated least-privilege database account for normal operation rather than a MySQL administrator account.

The generated names are synthetic, but source attributes are sampled from the IBM HR attrition dataset. Keep source files and any production employee data in approved storage. Do not expose database dumps, credentials, or employee-level extracts in screenshots or public repositories.

Implementation Limitations

- The application code does not define authentication or role-based authorization. Restrict access to trusted users and a controlled local environment.
- The sample records and derived reviews are synthetic. They are suitable for functional demonstrations, not personnel decisions or actual workforce reporting.
- The attrition view reports an observed source label by employee category; it is not an individual risk score.
- SCD Type 2 tracks department, job role, job level, and monthly income. It does not record every employee field or identify the user who made each change.
- No automated scheduler is defined in the repository. Data generation, SQL loads, and warehouse refreshes are initiated.
- The source schema does not specify a currency code for monthly income.

15 Repository Reference

Path	Purpose
app/streamlit_app.py	Home analytics page.
app/pages/1_Employees.py	Employee onboarding, search, editing, and history.
app/pages/2_Projects.py	Project creation, assignments, and project listing.
app/pages/3_Reviews.py	Performance-review entry and recent review listing.
app/pages/4_Dashboard.py	Filtered analytics dashboard.
app/helpers.py and app/styles.py	Shared UI helpers, display labels, page styling, and chart utilities.
src/entities.py	Validated Employee, Project, and Review entities.
src/managers.py	Application operations and analytics SQL.
src/db_manager.py and src/config.py	MySQL connection pool, query helpers, and .env configuration.
src/etl/data_synthesizer.py	Synthetic employee history and activity generation.
src/etl/staging_loader.py and run_staging_load.py	Batch-aware staging writes and pipeline entry point.
sql/00_create_schemas.sql through sql/07_etl_olap.sql	Schema creation, loading procedures, and warehouse ETL.
docs/*.png	Architecture, OLTP, staging, and OLAP diagrams reproduced in this documentation.

Glossary

Term	Meaning
ETL	Extract, transform, and load; the process that moves and shapes data for its next storage layer.
Fact table	A table of measurable business events. Here, each fact row represents one performance review.
OLTP	Online transaction processing; the normalized operational schema used by application forms.
OLAP	Online analytical processing; the dimensional warehouse used for aggregation and reporting.
SCD Type 2	A dimension-history method that expires an old row and inserts a new version when tracked attributes change.
Surrogate key	A generated warehouse key that identifies a dimension row independently of its source business ID.
Business key	A stable identifier from the operational domain, such as employee_id or project_code.
Synthetic data	Generated records that imitate source patterns for software demonstration and analysis development.

Project Summary

The project links a Streamlit HR workbench to a MySQL operational database and a review-centered analytics warehouse. The staging loader creates a 100,000-employee synthetic demonstration dataset from the IBM HR source file. SQL procedures move data through staging and OLTP into a dimensional model, where

employee history is preserved with SCD Type 2 and each review is associated with the employee version valid on its date.